

School of Future Tech

ITM Skills University



Mini Project – Report

Case Study / Problem Statement 73 |

on

AutoServe

Car Service Center Job Card Management System

Database Management Systems — SQL & NoSQL

by

Subrata Bhaktipad Panda — 150096725110

B.Tech Computer Science Engineering and AI | Semester III

Cohort: Larry Page

Academic Year 2025–2029

Report

PostgreSQL | ER Modelling | Normalization (2NF → BCNF) | SQL Views

Index

No.	Section	Page
1	Abstract	3
2	Introduction	3
3	Objectives	3
4	Problem Statement	3
5	Entities and Business Rules	4
6	Tools & Technologies	4
7	ER Diagram	5
8	Relational Schema & Normalization	5
9	DDL Scripts — Table Creation	7
10	DML Scripts — Sample Data	9
11	Query Set	11
12	View — PendingJobCards	14
13	Referential Integrity Demonstration	17
14	Results and Conclusion	18

1. Abstract

AutoServe is a relational database designed for a car service centre. It tracks every vehicle's service job card — including the labour charged and the spare parts consumed — so that the total job cost is computed automatically and mechanics' workloads can be reviewed at any time.

The project covers the complete database design lifecycle: an ER model of the five core entities (Vehicle, Customer, Mechanic, ServiceJob and SparePart), conversion into a normalized relational schema (2NF through 3NF/BCNF), DDL with primary key, foreign key, NOT NULL, UNIQUE, CHECK and DEFAULT constraints, DML with realistic sample records, analytical SELECT queries using JOINS, subqueries and aggregate functions, and a **PendingJobCards** view that acts as a live workload dashboard for the service manager.

2. Introduction

A car service centre handles many vehicles at once. Each vehicle arrives with a job card that records who worked on it, what was done, which parts were fitted and how much the customer must pay. When this information lives on paper or in loosely organised spreadsheets, three problems appear quickly: part prices are copied onto every job and drift out of date, totals are calculated by hand and contain errors, and the service manager has no quick way to see which jobs are still open or how evenly work is spread across mechanics.

AutoServe solves these problems with a normalized relational design. Every fact is stored exactly once — a part's price lives only in the SparePart table, a vehicle's owner only in the Vehicle table — and derived values such as the total job cost are calculated by queries rather than stored. Referential integrity guarantees that every job card links to a valid vehicle (and through it a valid customer) and a valid mechanic.

3. Objectives

- **ER Diagram:** identify the entities Vehicle, Customer, Mechanic, ServiceJob and SparePart and draw an ER diagram with relationships and clearly marked primary and foreign keys.
- **Schema Design & Normalization:** convert the ER diagram into a relational schema and remove the spare-part price redundancy (2NF) by keeping SparePart as the single source of price and referencing it by PartID from a job-parts table.
- **SQL Queries:** write DDL with primary/foreign key and NOT NULL constraints, DML with realistic sample data, and a query that computes the total cost (labour plus the sum of spare-part costs) for each job using a JOIN and SUM aggregation.
- **Advanced Concept (View):** create the view PendingJobCards joining ServiceJob, Vehicle and Mechanic to show every job not yet marked complete.
- **Reporting:** report revenue generated per mechanic with a GROUP BY / SUM query and find above-average jobs with a subquery.

4. Problem Statement

AutoServe wants a database for a car service centre that tracks each vehicle's service job card, including labour and spare parts used, so the total job cost is computed automatically and mechanics' workloads can be reviewed. The ServiceJob table must include the fields JobID, VehicleID, MechanicID, JobDate, LabourCharge and Status. Spare-part price is currently repeated on every service-job row; SparePart must become the single source of price and be referenced by PartID from a job-parts table.

Expected outcomes: total job cost (parts + labour) computed automatically for every job; an always-current PendingJobCards view of outstanding work; revenue per mechanic through a GROUP BY SUM query; and referential integrity linking every job card to a valid vehicle, customer and mechanic.

5. Entities and Business Rules

5.1 Entities

Entity	Description	Key attributes
Customer	Person who owns one or more vehicles	CustomerID (PK), Name, Phone, Email, Address
Vehicle	A car brought in for service	VehicleID (PK), RegNo (unique), Make, Model, MfgYear, CustomerID (FK)
Mechanic	Technician assigned to job cards	MechanicID (PK), Name, Phone, Specialization
ServiceJob	One job card for one vehicle	JobID (PK), VehicleID (FK), MechanicID (FK), JobDate, LabourCharge, Status
SparePart	Catalogue of parts with the single source of price	PartID (PK), PartName, UnitPrice, StockQty
JobParts	Junction table: parts used on each job	(JobID, PartID) composite PK, Quantity

5.2 Business Rules

- A customer can own many vehicles; every vehicle belongs to exactly one customer.
- A vehicle can have many service jobs over time; each job card is for exactly one vehicle.
- Each job card is assigned to exactly one mechanic; a mechanic handles many jobs.
- A job can use many spare parts and a spare part can be used on many jobs (many-to-many, resolved by JobParts). A job may also use no parts at all (labour-only).
- A job's Status is one of 'Pending', 'In Progress' or 'Completed' and defaults to 'Pending'.
- Labour charge cannot be negative, unit price must be positive, stock cannot be negative and quantity used must be at least 1.
- The total job cost is never stored; it is always derived as $\text{LabourCharge} + \sum(\text{Quantity} \times \text{UnitPrice})$.

6. Tools & Technologies

Category	Tool	Purpose
DBMS	PostgreSQL	Relational database server used to create and run all scripts
SQL client	psql / pgAdmin 4	Execute DDL, DML and queries; capture output screenshots
Language	SQL (ANSI-style, PostgreSQL dialect)	DDL, DML, JOINS, subqueries, CTE, aggregates, views
ER diagram	Graphviz	Entity-relationship diagram in Section 7
Documentation	Microsoft Word	Project report

7. ER Diagram

The ER diagram below shows the entities, their attributes and the relationships among them. Primary keys are underlined and in bold; foreign keys are in *italics* and marked (FK). Cardinality is written at each end of the relationship line (1 or M).

AutoServe — Entity Relationship (ER) Diagram

Car Service Center Job Card Management System | Chen notation | attribute colour = the entity it belongs to

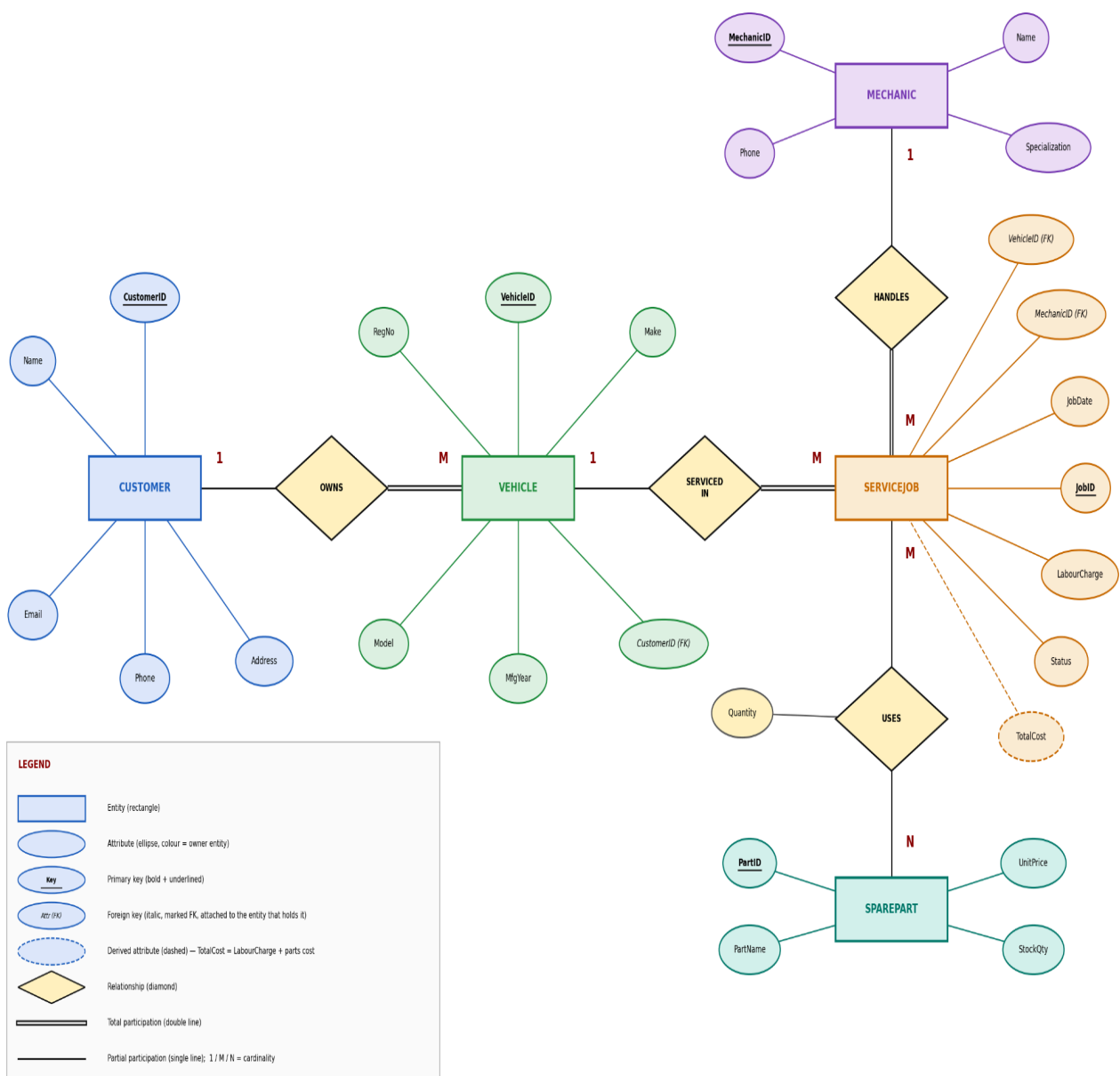


Figure — ER diagram of the AutoServe database

7.1 Relationships

Relationship	Entities	Cardinality	Implemented by
owns	Customer — Vehicle	1 : M	Vehicle.CustomerID → Customer
receives	Vehicle — ServiceJob	1 : M	ServiceJob.VehicleID → Vehicle
handles	Mechanic — ServiceJob	1 : M	ServiceJob.MechanicID → Mechanic
uses	ServiceJob — SparePart	M : N	Junction table JobParts (JobID, PartID)

No one-to-one relationship is needed in this domain. The many-to-many relationship between ServiceJob and SparePart is resolved into two one-to-many relationships through the junction table JobParts, whose composite primary key (JobID, PartID) also prevents the same part appearing twice on one job card. The customer of a job is reached through Vehicle, so it is not repeated on ServiceJob.

8. Relational Schema & Normalization

8.1 Relational Schema

Table	Schema (bold = primary key; → = foreign key)
Customer	Customer(CustomerID, Name, Phone, Email, Address)
Vehicle	Vehicle(VehicleID, RegNo, Make, Model, MfgYear, CustomerID → Customer)
Mechanic	Mechanic(MechanicID, Name, Phone, Specialization)
ServiceJob	ServiceJob(JobID, VehicleID → Vehicle, MechanicID → Mechanic, JobDate, LabourCharge, Status)
SparePart	SparePart(PartID, PartName, UnitPrice, StockQty)
JobParts	JobParts(JobID → ServiceJob, PartID → SparePart, Quantity)

Bold attributes form the primary key; “→” marks a foreign key and the table it references.

8.2 Normalization

Starting point (unnormalized design). The problem statement describes a design in which the spare-part details are repeated on every service-job row:

```
ServiceJob_UNF(JobID, VehicleID, MechanicID, JobDate, LabourCharge, Status,
               PartID, PartName, UnitPrice, Quantity)      PK = (JobID, PartID)
```

The functional dependencies in this table are:

- JobID → VehicleID, MechanicID, JobDate, LabourCharge, Status (depends on **part** of the key)
- PartID → PartName, UnitPrice, StockQty (depends on **part** of the key)
- (JobID, PartID) → Quantity (depends on the whole key)

Because non-key attributes depend on only part of the composite key, the table violates **2NF** and suffers from anomalies:

- **Update anomaly:** changing a part's price means editing every job row that used it; missing one leaves inconsistent prices.
- **Insertion anomaly:** a new part cannot be added to the catalogue until it is used on some job.
- **Deletion anomaly:** deleting the only job that used a part also deletes the part's name and price.
- **Redundancy:** job-level data (date, labour, status) is repeated for every part on the job, and a labour-only job has no natural row without NULL part columns.

1NF. All attributes are atomic, there are no repeating groups, and every row is identified by a primary key. Each table in the final design satisfies 1NF.

2NF. The partial dependencies are removed by decomposition: job attributes stay in ServiceJob (key JobID), part attributes move to SparePart (key PartID), and only Quantity — which depends on the full key — stays in the junction table JobParts (key JobID, PartID). SparePart is now the single source of price.

3NF / BCNF. In every table the only determinants are candidate keys, so no transitive dependencies remain and each table is in 3NF and BCNF:

- **Customer, Mechanic, SparePart, ServiceJob:** every non-key attribute depends only on the primary key.
- **Vehicle:** both VehicleID and RegNo (declared UNIQUE) are candidate keys and are the only determinants.
- **JobParts:** Quantity depends on the full composite key and nothing else.

Design note. In the real world Model → Make (a Swift is always a Maruti). Model names are not guaranteed unique across manufacturers, so a separate model catalogue is not enforced here; Make and Model are kept as plain attributes of Vehicle. The total job cost is deliberately not stored because it is derivable, which prevents it from becoming stale.

8.3 Constraints Summary

Constraint	Where applied
PRIMARY KEY	Every table; JobParts uses the composite key (JobID, PartID)
FOREIGN KEY	Vehicle.CustomerID, ServiceJob.VehicleID, ServiceJob.MechanicID, JobParts.JobID, JobParts.PartID
NOT NULL	Name/Phone (Customer, Mechanic), RegNo/Make/Model, JobDate, LabourCharge, Status, PartName, UnitPrice, StockQty, Quantity and all foreign keys
UNIQUE	Vehicle.RegNo
CHECK	MfgYear ≥ 1980; LabourCharge ≥ 0; Status IN (Pending, In Progress, Completed); UnitPrice > 0; StockQty ≥ 0; Quantity > 0
DEFAULT	ServiceJob.Status = 'Pending'; SparePart.StockQty = 0

9. DDL Scripts — Table Creation

Tables are created parent-first (Customer, Vehicle, Mechanic, ServiceJob, SparePart, JobParts) so that every foreign key refers to a table that already exists.

```

CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    Name        VARCHAR(100) NOT NULL,
    Phone       VARCHAR(15)  NOT NULL,
    Email       VARCHAR(100),
    Address     VARCHAR(200)
);

CREATE TABLE Vehicle (
    VehicleID   INT PRIMARY KEY,
    RegNo       VARCHAR(15) NOT NULL UNIQUE,
    Make        VARCHAR(50) NOT NULL,
    Model       VARCHAR(50) NOT NULL,
    MfgYear     INT CHECK (MfgYear >= 1980),
    CustomerID  INT NOT NULL REFERENCES Customer(CustomerID)
);

CREATE TABLE Mechanic (
    MechanicID  INT PRIMARY KEY,
    Name        VARCHAR(100) NOT NULL,
    Phone       VARCHAR(15)  NOT NULL,
    Specialization VARCHAR(50)
);

CREATE TABLE ServiceJob (
    JobID       INT PRIMARY KEY,
    VehicleID   INT NOT NULL REFERENCES Vehicle(VehicleID),
    MechanicID  INT NOT NULL REFERENCES Mechanic(MechanicID),
    JobDate     DATE NOT NULL,
    LabourCharge NUMERIC(10,2) NOT NULL CHECK (LabourCharge >= 0),
    Status      VARCHAR(20) NOT NULL DEFAULT 'Pending'
                CHECK (Status IN ('Pending', 'In Progress', 'Completed'))
);

CREATE TABLE SparePart (
    PartID      INT PRIMARY KEY,
    PartName    VARCHAR(100) NOT NULL,
    UnitPrice   NUMERIC(10,2) NOT NULL CHECK (UnitPrice > 0),
    StockQty    INT NOT NULL DEFAULT 0 CHECK (StockQty >= 0)
);

CREATE TABLE JobParts (
    JobID       INT NOT NULL REFERENCES ServiceJob(JobID),
    PartID      INT NOT NULL REFERENCES SparePart(PartID),
    Quantity    INT NOT NULL CHECK (Quantity > 0),
    PRIMARY KEY (JobID, PartID)
);

```

```

autoserve=# CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    Name       VARCHAR(100) NOT NULL,
    Phone      VARCHAR(15)  NOT NULL,
    Email      VARCHAR(100),
    Address    VARCHAR(200)
);

CREATE TABLE Vehicle (
    VehicleID  INT PRIMARY KEY,
    RegNo      VARCHAR(15) NOT NULL UNIQUE,
    Make       VARCHAR(50) NOT NULL,
    Model      VARCHAR(50) NOT NULL,
    MfgYear    INT CHECK (MfgYear >= 1980),
    CustomerID INT NOT NULL REFERENCES Customer(CustomerID)
);

CREATE TABLE Mechanic (
    MechanicID INT PRIMARY KEY,
    Name       VARCHAR(100) NOT NULL,
    Phone      VARCHAR(15)  NOT NULL,
    Specialization VARCHAR(50)
);

CREATE TABLE ServiceJob (
    JobID      INT PRIMARY KEY,
    VehicleID  INT NOT NULL REFERENCES Vehicle(VehicleID),
    MechanicID INT NOT NULL REFERENCES Mechanic(MechanicID),
    JobDate    DATE NOT NULL,
    LabourCharge NUMERIC(10,2) NOT NULL CHECK (LabourCharge >= 0),
    Status     VARCHAR(20) NOT NULL DEFAULT 'Pending'
              CHECK (Status IN ('Pending', 'In Progress', 'Completed'))
);

CREATE TABLE SparePart (
    PartID     INT PRIMARY KEY,
    PartName   VARCHAR(100) NOT NULL,
    UnitPrice  NUMERIC(10,2) NOT NULL CHECK (UnitPrice > 0),
    StockQty   INT NOT NULL DEFAULT 0 CHECK (StockQty >= 0)
);

CREATE TABLE JobParts (
    JobID      INT NOT NULL REFERENCES ServiceJob(JobID),
    PartID     INT NOT NULL REFERENCES SparePart(PartID),
    Quantity   INT NOT NULL CHECK (Quantity > 0),
    PRIMARY KEY (JobID, PartID)
);
[CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE

```

Figure — DDL execution output

```
[autoserve=# \dt
              List of tables
 Schema |      Name      | Type  |      Owner
-----+-----+-----+-----
 public | customer       | table | subratapanda
 public | jobparts       | table | subratapanda
 public | mechanic       | table | subratapanda
 public | servicejob     | table | subratapanda
 public | sparepart      | table | subratapanda
 public | vehicle        | table | subratapanda
(6 rows)

autoserve=#
```

Figure — Tables created and ServiceJob structure with constraints

10. DML Scripts — Sample Data

The sample data contains 4 customers, 6 vehicles, 3 mechanics, 8 spare parts, 8 service jobs and 15 job-part rows. Jobs span all three statuses, and job 1008 is deliberately labour-only (no parts) to prove that the cost query handles jobs without parts correctly.

```

INSERT INTO Customer VALUES
(1, 'Rahul Sharma', '9820011111', 'rahul@example.com', 'Andheri, Mumbai'),
(2, 'Priya Patel', '9820022222', 'priya@example.com', 'Bandra, Mumbai'),
(3, 'Amit Verma', '9820033333', 'amit@example.com', 'Thane'),
(4, 'Sneha Iyer', '9820044444', 'sneha@example.com', 'Powai, Mumbai');

INSERT INTO Vehicle VALUES
(101, 'MH02AB1234', 'Maruti', 'Swift', 2019, 1),
(102, 'MH04CD5678', 'Hyundai', 'i20', 2021, 2),
(103, 'MH01EF9012', 'Honda', 'City', 2018, 1),
(104, 'MH12GH3456', 'Tata', 'Nexon', 2022, 3),
(105, 'MH03JK7890', 'Toyota', 'Innova', 2017, 4),
(106, 'MH02LM2345', 'Mahindra', 'XUV700', 2023, 2);

INSERT INTO Mechanic VALUES
(1, 'Ramesh Kulkarni', '9900011111', 'Engine'),
(2, 'Sunil Yadav', '9900022222', 'Electrical'),
(3, 'Imran Shaikh', '9900033333', 'Brakes & Suspension');

INSERT INTO SparePart VALUES
(1, 'Engine Oil 5W-30 (4L)', 1800.00, 40),
(2, 'Oil Filter', 350.00, 60),
(3, 'Air Filter', 450.00, 50),
(4, 'Brake Pad Set', 2200.00, 25),
(5, 'Spark Plug', 400.00, 100),
(6, 'Battery 12V', 5500.00, 10),
(7, 'Wiper Blade', 300.00, 45),
(8, 'Clutch Plate', 3500.00, 12);

INSERT INTO ServiceJob VALUES
(1001, 101, 1, '2026-09-01', 1200.00, 'Completed'),
(1002, 102, 2, '2026-09-03', 800.00, 'Completed'),
(1003, 103, 3, '2026-09-05', 1500.00, 'Completed'),
(1004, 104, 1, '2026-09-10', 1000.00, 'Completed'),
(1005, 105, 3, '2026-09-15', 2500.00, 'In Progress'),
(1006, 106, 2, '2026-09-18', 900.00, 'Pending'),
(1007, 101, 2, '2026-09-25', 700.00, 'In Progress'),
(1008, 102, 1, '2026-09-28', 600.00, 'Pending');

INSERT INTO JobParts VALUES
(1001, 1, 1), (1001, 2, 1), (1001, 3, 1),
(1002, 6, 1),
(1003, 4, 1), (1003, 7, 2),
(1004, 1, 1), (1004, 2, 1), (1004, 5, 4),
(1005, 8, 1), (1005, 4, 1),
(1006, 3, 1), (1006, 7, 2),
(1007, 5, 4), (1007, 2, 1);
-- Job 1008 is labour-only (no parts) on purpose

```

```

autoserve=# INSERT INTO Customer VALUES
(1, 'Rahul Sharma', '9820011111', 'rahul@example.com', 'Andheri, Mumbai'),
(2, 'Priya Patel', '9820022222', 'priya@example.com', 'Bandra, Mumbai'),
(3, 'Amit Verma', '9820033333', 'amit@example.com', 'Thane'),
(4, 'Sneha Iyer', '9820044444', 'sneha@example.com', 'Powai, Mumbai');

INSERT INTO Vehicle VALUES
(101, 'MH02AB1234', 'Maruti', 'Swift', 2019, 1),
(102, 'MH04CD5678', 'Hyundai', 'i20', 2021, 2),
(103, 'MH01EF9012', 'Honda', 'City', 2018, 1),
(104, 'MH12GH3456', 'Tata', 'Nexon', 2022, 3),
(105, 'MH03JK7890', 'Toyota', 'Innova', 2017, 4),
(106, 'MH02LM2345', 'Mahindra', 'XUV700', 2023, 2);

INSERT INTO Mechanic VALUES
(1, 'Ramesh Kulkarni', '9900011111', 'Engine'),
(2, 'Sunil Yadav', '9900022222', 'Electrical'),
(3, 'Imran Shaikh', '9900033333', 'Brakes & Suspension');

INSERT INTO SparePart VALUES
(1, 'Engine Oil 5W-30 (4L)', 1800.00, 40),
(2, 'Oil Filter', 350.00, 60),
(3, 'Air Filter', 450.00, 50),
(4, 'Brake Pad Set', 2200.00, 25),
(5, 'Spark Plug', 400.00, 100),
(6, 'Battery 12V', 5500.00, 10),
(7, 'Wiper Blade', 300.00, 45),
(8, 'Clutch Plate', 3500.00, 12);

INSERT INTO ServiceJob VALUES
(1001, 101, 1, '2026-09-01', 1200.00, 'Completed'),
(1002, 102, 2, '2026-09-03', 800.00, 'Completed'),
(1003, 103, 3, '2026-09-05', 1500.00, 'Completed'),
(1004, 104, 1, '2026-09-10', 1000.00, 'Completed'),
(1005, 105, 3, '2026-09-15', 2500.00, 'In Progress'),
(1006, 106, 2, '2026-09-18', 900.00, 'Pending'),
(1007, 101, 2, '2026-09-25', 700.00, 'In Progress'),
(1008, 102, 1, '2026-09-28', 600.00, 'Pending');

INSERT INTO JobParts VALUES
(1001, 1, 1), (1001, 2, 1), (1001, 3, 1),
(1002, 6, 1),
(1003, 4, 1), (1003, 7, 2),
(1004, 1, 1), (1004, 2, 1), (1004, 5, 4),
(1005, 8, 1), (1005, 4, 1),
(1006, 3, 1), (1006, 7, 2),
(1007, 5, 4), (1007, 2, 1);
[INSERT 0 4
INSERT 0 6
INSERT 0 3
INSERT 0 8
INSERT 0 8
INSERT 0 15

```

Figure — Successful Insertion

11. Query Set

11.1 Total Job Cost (JOIN + SUM)

Purpose: compute the total cost of every service job as labour charge plus the sum of Quantity × UnitPrice over its parts. Two LEFT JOINs link ServiceJob to JobParts and then to SparePart, so the price always comes from the single source (SparePart). LEFT JOIN keeps job 1008, which has no parts, and COALESCE turns its NULL parts sum into 0.

```
SELECT sj.JobID,
       sj.LabourCharge,
       COALESCE(SUM(jp.Quantity * sp.UnitPrice), 0) AS parts_cost,
       sj.LabourCharge + COALESCE(SUM(jp.Quantity * sp.UnitPrice), 0) AS total_job_cost
FROM ServiceJob sj
LEFT JOIN JobParts jp ON sj.JobID = jp.JobID
LEFT JOIN SparePart sp ON jp.PartID = sp.PartID
GROUP BY sj.JobID, sj.LabourCharge
ORDER BY sj.JobID;
```

```
autoserve=# SELECT sj.JobID,
       sj.LabourCharge,
       COALESCE(SUM(jp.Quantity * sp.UnitPrice), 0) AS parts_cost,
       sj.LabourCharge + COALESCE(SUM(jp.Quantity * sp.UnitPrice), 0) AS total_job_cost
FROM ServiceJob sj
LEFT JOIN JobParts jp ON sj.JobID = jp.JobID
LEFT JOIN SparePart sp ON jp.PartID = sp.PartID
GROUP BY sj.JobID, sj.LabourCharge
ORDER BY sj.JobID;
 jobid | labourcharge | parts_cost | total_job_cost
-----+-----+-----+-----
 1001 |    1200.00 |    2600.00 |    3800.00
 1002 |     800.00 |    5500.00 |    6300.00
 1003 |    1500.00 |    2800.00 |    4300.00
 1004 |    1000.00 |    3750.00 |    4750.00
 1005 |    2500.00 |    5700.00 |    8200.00
 1006 |     900.00 |    1050.00 |    1950.00
 1007 |     700.00 |    1950.00 |    2650.00
 1008 |     600.00 |         0 |     600.00
(8 rows)
```

Figure — Total job cost query output

Expected output

jobid	labourcharge	parts_cost	total_job_cost
1001	1200.00	2600.00	3800.00
1002	800.00	5500.00	6300.00
1003	1500.00	2800.00	4300.00
1004	1000.00	3750.00	4750.00
1005	2500.00	5700.00	8200.00
1006	900.00	1050.00	1950.00
1007	700.00	1950.00	2650.00
1008	600.00	0.00	600.00

11.2 Revenue per Mechanic (GROUP BY + SUM)

Purpose: report how many jobs each mechanic has handled and the revenue those jobs represent. The derived table `jt` calculates each job's total cost (as in 11.1); the outer query then groups by mechanic with COUNT and SUM.

```
SELECT m.MechanicID,
       m.Name,
       COUNT(jt.JobID)      AS jobs_handled,
       SUM(jt.total_cost)   AS revenue
FROM Mechanic m
JOIN (
  SELECT sj.JobID, sj.MechanicID,
         sj.LabourCharge + COALESCE(SUM(jp.Quantity * sp.UnitPrice), 0) AS total_cost
  FROM ServiceJob sj
  LEFT JOIN JobParts jp ON sj.JobID = jp.JobID
  LEFT JOIN SparePart sp ON jp.PartID = sp.PartID
  GROUP BY sj.JobID, sj.MechanicID, sj.LabourCharge
) jt ON m.MechanicID = jt.MechanicID
GROUP BY m.MechanicID, m.Name
ORDER BY revenue DESC;
```

```
autoserve=# SELECT m.MechanicID,
       m.Name,
       COUNT(jt.JobID)      AS jobs_handled,
       SUM(jt.total_cost)   AS revenue
FROM Mechanic m
JOIN (
  SELECT sj.JobID, sj.MechanicID,
         sj.LabourCharge + COALESCE(SUM(jp.Quantity * sp.UnitPrice), 0) AS total_cost
  FROM ServiceJob sj
  LEFT JOIN JobParts jp ON sj.JobID = jp.JobID
  LEFT JOIN SparePart sp ON jp.PartID = sp.PartID
  GROUP BY sj.JobID, sj.MechanicID, sj.LabourCharge
) jt ON m.MechanicID = jt.MechanicID
GROUP BY m.MechanicID, m.Name
ORDER BY revenue DESC;
```

mechanicid	name	jobs_handled	revenue
3	Imran Shaikh	2	12500.00
2	Sunil Yadav	3	10900.00
1	Ramesh Kulkarni	3	9150.00

(3 rows)

Figure — Revenue per mechanic query output

Expected output

mechanicid	name	jobs_handled	revenue
3	Imran Shaikh	2	12500.00
2	Sunil Yadav	3	10900.00
1	Ramesh Kulkarni	3	9150.00

Note: this query counts every job card, including Pending and In Progress jobs, so “revenue” is the billed value of all jobs. To count only finished work, add `WHERE sj.Status = 'Completed'` inside the derived table.

11.3 Jobs Costing More than the Average (Subquery + CTE)

Purpose: list the jobs whose total cost is above the average job cost. A common table expression (`job_totals`) computes each job's total once and the scalar subquery `SELECT AVG(total_cost)` supplies the average (4068.75 for this data).

```

WITH job_totals AS (
    SELECT sj.JobID,
           sj.LabourCharge + COALESCE(SUM(jp.Quantity * sp.UnitPrice), 0) AS total_cost
    FROM ServiceJob sj
    LEFT JOIN JobParts jp ON sj.JobID = jp.JobID
    LEFT JOIN SparePart sp ON jp.PartID = sp.PartID
    GROUP BY sj.JobID, sj.LabourCharge
)
SELECT JobID, total_cost
FROM job_totals
WHERE total_cost > (SELECT AVG(total_cost) FROM job_totals)
ORDER BY total_cost DESC;

```

```

autoserve=# WITH job_totals AS (
    SELECT sj.JobID,
           sj.LabourCharge + COALESCE(SUM(jp.Quantity * sp.UnitPrice), 0) AS total_cost
    FROM ServiceJob sj
    LEFT JOIN JobParts jp ON sj.JobID = jp.JobID
    LEFT JOIN SparePart sp ON jp.PartID = sp.PartID
    GROUP BY sj.JobID, sj.LabourCharge
)
SELECT JobID, total_cost
FROM job_totals
WHERE total_cost > (SELECT AVG(total_cost) FROM job_totals)
ORDER BY total_cost DESC;
 jobid | total_cost
-----+-----
  1005 |    8200.00
  1002 |    6300.00
  1004 |    4750.00
  1003 |    4300.00
(4 rows)

```

Figure 11.3 — Above-average job cost query output

Expected output

jobid	total_cost
1005	8200.00
1002	6300.00
1004	4750.00
1003	4300.00

12. View — PendingJobCards

The view joins ServiceJob, Vehicle and Mechanic and keeps every job whose status is not 'Completed'. Because a view stores the query rather than the data, it always reflects the current contents of the base tables — giving the service manager a live workload dashboard.

12.1 View Creation

```
CREATE OR REPLACE VIEW PendingJobCards AS
SELECT sj.JobID,
       sj.JobDate,
       sj.Status,
       v.RegNo,
       v.Make,
       v.Model,
       m.MechanicID,
       m.Name AS mechanic_name,
       sj.LabourCharge
FROM ServiceJob sj
JOIN Vehicle v ON sj.VehicleID = v.VehicleID
JOIN Mechanic m ON sj.MechanicID = m.MechanicID
WHERE sj.Status <> 'Completed';
```

```
autoserve=# CREATE OR REPLACE VIEW PendingJobCards AS
SELECT sj.JobID,
       sj.JobDate,
       sj.Status,
       v.RegNo,
       v.Make,
       v.Model,
       m.MechanicID,
       m.Name AS mechanic_name,
       sj.LabourCharge
FROM ServiceJob sj
JOIN Vehicle v ON sj.VehicleID = v.VehicleID
JOIN Mechanic m ON sj.MechanicID = m.MechanicID
WHERE sj.Status <> 'Completed';
CREATE VIEW
```

Figure— View creation

12.2 Querying the View

```
SELECT * FROM PendingJobCards ORDER BY JobDate;
```

```
autoserve=# SELECT * FROM PendingJobCards ORDER BY JobDate;
 jobid | jobdate | status | regno | make | model | mechanicid | mechanic_name | labourcharge
-----|-----|-----|-----|-----|-----|-----|-----|-----
  1005 | 2026-09-15 | In Progress | MH03JK7890 | Toyota | Innova | 3 | Imran Shaikh | 2500.00
  1006 | 2026-09-18 | Pending | MH02LM2345 | Mahindra | XUV700 | 2 | Sunil Yadav | 900.00
  1007 | 2026-09-25 | In Progress | MH02AB1234 | Maruti | Swift | 2 | Sunil Yadav | 700.00
  1008 | 2026-09-28 | Pending | MH04CD5678 | Hyundai | i20 | 1 | Ramesh Kulkarni | 600.00
(4 rows)
```

Figure — Open job cards from the view

Expected output

jobid	jobdate	status	regno	make	model	mechanic_name	labour
1005	2026-09-15	In Progress	MH03JK7890	Toyota	Innova	Imran Shaikh	2500.00

jobid	jobdate	status	regno	make	model	mechanic_name	labour
1006	2026-09-18	Pending	MH02LM2345	Mahindra	XUV700	Sunil Yadav	900.00
1007	2026-09-25	In Progress	MH02AB1234	Maruti	Swift	Sunil Yadav	700.00
1008	2026-09-28	Pending	MH04CD5678	Hyundai	i20	Ramesh Kulkarni	600.00

The MechanicID column of the view is omitted from the expected table above for width; it appears in your actual output.

12.3 Open Jobs per Mechanic (Workload Review)

Querying the view with GROUP BY shows how many unfinished jobs each mechanic currently holds.

```
SELECT mechanic_name, COUNT(*) AS open_jobs
FROM PendingJobCards
GROUP BY mechanic_name
ORDER BY open_jobs DESC;
```

```
autoserve=# SELECT mechanic_name, COUNT(*) AS open_jobs
FROM PendingJobCards
GROUP BY mechanic_name
[ORDER BY open_jobs DESC;
 mechanic_name | open_jobs
-----+-----
Sunil Yadav    |      2
Ramesh Kulkarni |      1
Imran Shaikh   |      1
(3 rows)
```

Figure — Workload per mechanic from PendingJobCards

Expected output

mechanic_name	open_jobs
Sunil Yadav	2
Imran Shaikh	1
Ramesh Kulkarni	1

Imran Shaikh and Ramesh Kulkarni tie at 1 open job, so their relative order may differ.

12.4 Live Update Demonstration

Marking job 1006 as Completed removes it from the view immediately, without any change to the view itself.

```
UPDATE ServiceJob SET Status = 'Completed' WHERE JobID = 1006;
SELECT JobID FROM PendingJobCards; -- 1006 has disappeared
```

```

autoserve=# UPDATE ServiceJob SET Status = 'Completed' WHERE JobID = 1006;
[SELECT JobID FROM PendingJobCards;  -- 1006 has disappeared
UPDATE 1
  jobid
-----
   1005
   1007
   1008
(3 rows)

```

Figure — Job 1006 no longer appears in PendingJobCards

Expected output

jobid
1005
1007
1008

Run this step last, because it changes the data. To restore the original state afterwards: `UPDATE ServiceJob SET Status = 'Pending' WHERE JobID = 1006;`

13. Referential Integrity Demonstration

To prove that every job card must link to a valid vehicle, the script below tries to insert a job for vehicle 999, which does not exist. The foreign key constraint on ServiceJob.VehicleID rejects the row, so no orphan job card can enter the database.

```

INSERT INTO ServiceJob VALUES (1009, 999, 1, '2026-09-30', 500, 'Pending');
-- ERROR: insert or update on table "servicejob" violates foreign key
-- constraint "servicejob_vehicleid_fkey"

```

```

autoserve=# INSERT INTO ServiceJob VALUES (1009, 999, 1, '2026-09-30', 500, 'Pending');
-- ERROR: insert or update on table "servicejob" violates foreign key
-- constraint "servicejob_vehicleid_fkey"
ERROR:  insert or update on table "servicejob" violates foreign key constraint "servicejob_vehicleid_fkey"
DETAIL:  Key (vehicleid)=(999) is not present in table "vehicle".

```

Figure— Foreign key violation rejected by the database

The same mechanism protects the other relationships: a vehicle cannot reference a missing customer, a job cannot reference a missing mechanic, and a job-part row cannot reference a missing job or part. Because the customer is reached through the vehicle, a valid vehicle also guarantees a valid customer.

14. Results and Conclusion

14.1 Findings

Expected outcome	How it was achieved	Evidence
Total job cost computed automatically	LEFT JOIN of ServiceJob, JobParts, SparePart with SUM and COALESCE	Figure 11.1
Always-current view of outstanding work	View PendingJobCards (Status <> 'Completed')	Figures 12.2, 12.4
Revenue per mechanic	GROUP BY mechanic with SUM over per-job totals	Figure 11.2
Referential integrity	Foreign keys on Vehicle, ServiceJob and JobParts	Figure 13.1
Price redundancy removed (2NF)	SparePart holds UnitPrice; JobParts references PartID	Section 8.2

- All six tables were created and populated without constraint errors.
- Labour-only job 1008 was handled correctly (parts cost 0, total 600.00) thanks to LEFT JOIN and COALESCE.
- Completing a job removed it from PendingJobCards instantly.
- An insert referencing a non-existent vehicle was rejected by the foreign key constraint.

14.2 Limitations

- Total cost is computed at query time; it is not stored on an invoice, so a later price change in SparePart would change the cost of past jobs. A real system would snapshot the price at billing time.
- Stock levels (StockQty) are not decremented automatically when parts are added to a job; this would need a trigger or application logic.
- There is no invoice/payment entity, no appointment scheduling and no user authentication.
- Revenue per mechanic includes jobs that are not yet completed.

14.3 Conclusion

AutoServe shows that a clean, normalized schema lets a service centre compute job costs, review mechanic workloads and keep its data consistent with plain SQL. Removing the repeated spare-part price (2NF), enforcing constraints in the database itself, and exposing open work through a view gives the service manager accurate, always-current information without any application code.